KANN AUDITS

Security Review
For HyperLend

Prepared For: **HyperLend**

Prepared By: **Kann, sa1nt, dev_razkky, derastephh, yusufthebdev, heeze**

Date Reported: **May 26, 2026**

Audit Summary

HyperLend engaged Kann Audits to perform a security review of their Leveraged Lending contracts. From April 4 to April 10, a team of security researchers performed a manual review of the in-scope source code and additionally triaged findings generated by the Kann AI Labs auditing tool. All findings identified during the assessment have been documented in the following report.

Protocol Overiew

HyperLend is a high-performance lending protocol on Hyperliquid, built for capital efficiency. It offers real-time leverage, dynamic rates, and deep liquidity access. Built for traders, quants, and market makers in need of efficient lending.

Scope

Repository: https://github.com/hyperlendx/looping-contracts/

Audited Commit: 4f5deaf

Final Commit: 4f5deaf

Files:

- src/StrategyManagerFactory.sol
- src/StrategyManager.sol
- src/Looping.sol
- src/periphery/GluexAdapter.sol
- src/periphery/LiquidSwapAdapter.sol

Methodology

An engagement with Kann Audits involves a team of security researchers performing independent reviews of the codebase at the start of the engagement, followed by collaborative review sessions where auditors share findings and explore additional attack vectors to improve overall coverage.

The review included a line-by-line manual inspection of the in-scope codebase, along with analysis of the protocol architecture, access control mechanisms, and state transitions. The team also assessed the code against common smart contract security best practices and industry standards.

The auditing process specifically considered:

- Testing against common and uncommon attack vectors
- Ensuring compliance with industry best practices and known smart contract security standards
- Verifying contract logic matches the client's intended behavior
- Cross-referencing patterns with industry-standard implementations
- Manual line-by-line review by experienced auditors
- Access control verification
- Architecture and design analysis to ensure secure component interactions and clearly defined trust boundaries
- State transition and logic review to detect inconsistencies, unexpected behavior, or potential exploitation paths

Risk Classification

- **High** – result in directly exploitable vulnerabilities that can lead to significant material loss or critical impact on the system and require immediate fixing.
- **Medium** – result in moderate loss of funds or impact multiple users, but only under specific conditions. They are exploitable without privileged access but typically require certain assumptions or edge cases.
- **Low/Info** – issues are non-exploitable findings that do not pose a direct security risk or impact the system's integrity. These issues are typically cosmetic, best-practice deviations, or related to documentation and compliance. While they do not require urgent remediation, they may be addressed to improve code quality and maintainability.

Findings Count

High	Medium	Low/Info
0	0	5

Detailed Findings

ID	Title	Severity	Status
L-01	Swap adapters wrap entire contract native balance which inflates swap output	Low	ACKNOWLEDGED
I-01	Redundant `block.number` check in transient storage	Info	RESOLVED
I-02	Closing a full position fails if the user has unrelated debt in the same asset	Info	ACKNOWLEDGED
I-03	StrategyManager cannot receive native token refunds despite native cleanup branch	Info	ACKNOWLEDGED
I-04	StrategyManagerFactory registry becomes stale after ownership transfer	Info	ACKNOWLEDGED

Swap Adapters Wrap Entire Contract Native Balance Which Inflates Swap Output

Low ACKNOWLEDGED

Description

In `swapExactTokensForTokensSupportingFeeOnTransferTokens`, when `tokenOut` is `WHYPE`, the adapter wraps the contract's entire native token balance into `WHYPE`:

```
if (address(this).balance > 0) {
    WHYPE.deposit{value: address(this).balance}();
}
```

As a result, any native tokens already present in the contract before the swap execution are also wrapped and included in the final `balanceOut` calculation together with the actual swap output.

This causes the output amount to be inflated since unrelated contract funds are accounted for as part of the current swap result.

```
function swapExactTokensForTokensSupportingFeeOnTransferTokens(
    ...
) external nonReentrant {
    ...

    (bool success, ) = gluex.call(gluexCallData);
    require(success, "GluexAdapter: gluex swap failed");

    if (address(this).balance > 0) {
        WHYPE.deposit{value: address(this).balance}();
    }

    uint256 balanceOut = IERC20(tokenOut).balanceOf(address(this));

    require(
        balanceOut >= amountOutMin,
        "GluexAdapter: minAmountOut > balanceOut"
    );

    IERC20(tokenOut).safeTransfer(to, balanceOut);
}
```

Impact

User receive more `balanceOut` than intended since unrelated native funds already present in the contract are included in the swap result and transferred as part of the output amount.

Redundant `block.number` check in transient storage

Info RESOLVED

Description

Both `LiquidSwapAdapter` and `GluexAdapter` store the current `block.number` in transient storage during `setSwapPath` and later verify that it matches during `swapExactTokens...`. The apparent intention is to ensure that the swap path is only valid if it was set within the same transaction as the swap execution.

```
function setSwapPath(
        address tokenIn,
        address tokenOut,
        bytes calldata gluexData
    ) external {
        // Generate unique slots for this token pair in transient storage
        bytes32 baseSlot = keccak256(abi.encodePacked(tokenIn, tokenOut));
        bytes32 blockSlot = keccak256(abi.encodePacked(baseSlot, "block"));

        assembly {
            tstore(blockSlot, number())
            tstore(baseSlot, gluexData.length)
        }

        // Store data in chunks of 32 bytes
        uint256 length = gluexData.length;
        for (uint256 i = 0; i < length; i += 32) {
            bytes32 chunk;
            assembly {
                chunk := calldataload(add(gluexData.offset, i))
                tstore(add(baseSlot, add(1, div(i, 32))), chunk)
            }
        }
    }
```

However, this additional `block.number` validation is redundant in the context of EIP-1153 transient storage. Under EIP-1153, transient storage is automatically cleared at the end of every transaction, not at the end of a block. As a result, any value written via `tstore` in one transaction cannot persist into a subsequent transaction, even if it occurs within the same block.

Therefore, the safety property the check attempts to enforce ensuring same transaction usage is already guaranteed by the EVM's transaction-scoped lifetime of transient storage. The comparison against `block.number` does not strengthen this guarantee and does not provide additional security beyond what EIP-1153 inherently enforces.

Closing a Full Position Fails if the User Has Unrelated Debt in the Same Asset

Info ACKNOWLEDGED

Description

In `Looping.sol`, users can close a leveraged position by providing the flashloan parameters and setting `withdrawAmount` to `type(uint256).max`. This signals the contract to fully unwind the position by reading the user's current yield token and debt token balances.

```
function _executeClosePosition(bytes memory params, address debtAsset) internal {
    (
        ...
    ) = abi.decode(...);

    IERC20 hYieldToken = IERC20(
        IPool(msg.sender).getReserveData(yieldAsset).aTokenAddress
    );

    // close full position if withdrawAmount == maxUint256
    if (withdrawAmount == type(uint256).max) {
        IERC20 debtDebtToken = IERC20(
            IPool(msg.sender)
                .getReserveData(debtAsset)
                .variableDebtTokenAddress
        );

        repaymentAmount = debtDebtToken.balanceOf(user);
        withdrawAmount = hYieldToken.balanceOf(user);
    }

    ...
}
```

However, `debtDebtToken.balanceOf(user)` returns the user's global debt balance for that specific asset across the entire lending pool. If the user has other active borrows using the same debt token (backed by different collateral assets outside of this specific looping strategy), `repaymentAmount` will be set to a value higher than the `flashloanAmount`. Because the `Looping` contract only holds the exact amount of `debtAsset` it flashloaned, the subsequent call to `IPool.repay` will revert due to insufficient balance.

If user opens first USDC/WETH position by supplying WETH then USDC/WBTC. Debt token is the same. If user wants to exit with the full WETH position, we wouldnt be able to since `repaymentAmount = debtDebtToken.balanceOf(user);` will take the debt balance from both positions.

To mitigate this we could specify the exact `repaymentAmount` amount instead of putting `uint256 max`. Or we could flashloan more amount so it repays all the debt but its not more than the collateral that will be swapped to the `debtAsset`. In this situation the flashloan amount < WETH. Then the second position is now without any debt so user can just call withdraw

StrategyManager Cannot Receive Native Token Refunds Despite Native Cleanup Branch

Info ACKNOWLEDGED

Relevant code

```
if (tokens[i] == address(0)){
        (bool sent,) = owner().call{value: address(this).balance}("");
        require(sent, "cleanOutTokens: failed to send native");
}
```

Description

`StrategyManager` contains logic for cleaning out native balances when `address(0)` is passed, which implies native-asset handling was considered. However, the contract does not implement `receive()` or a payable `fallback()`, so standard native refunds or direct native transfers to the contract revert.

StrategyManagerFactory Registry Becomes Stale After Ownership Transfer

Info ACKNOWLEDGED

Description

`StrategyManagerFactory` indexes strategy managers by the address that created them, but `StrategyManager` ownership is transferable after deployment.

Once a manager is transferred to a new owner, the factory registry is not updated. The original creator remains recorded as the owner of that tuple while the real new owner is invisible to the registry.

Disclaimers

A security audit can never guarantee the complete absence of vulnerabilities. Audits are a time, resource, and expertise-bound effort in which we aim to identify as many issues as possible.

About Kann Audits

Kann Audits is a top-tier Web3 security audit company, trusted by leading projects and providing comphrensive audits and expert guidance to secure the most critical blockchain protocols.

Kann Audits provides services such as manual auditing, AI audits using the Kann AI Labs tool, and incident response.

To learn more, visit https://kannaudits.com
To view our audit portfolio, visit https://github.com/KannAudits
To book an audit, message https://t.me/KannAudits